



Bases de Dados

AULA 11: Junções Internas, Junções Externas e Outros Tipos de Junção de Tabelas

Prof. Eduardo Corrêa Gonçalves
01/10/2026



I. Introdução

As junções são utilizadas em instruções SELECT para recuperar dados de duas tabelas e “juntá-los” com o objetivo de produzir uma saída combinada. As tabelas envolvidas em uma junção precisam compartilhar dados em comum através de um ou mais atributos. Na maioria das vezes (embora **não necessariamente**) estes atributos que “conectam” as tabelas estarão vinculados através de uma ligação Chave Primária-Chave Estrangeira.

Na primeira parte da matéria, aprendemos a trabalhar com o Produto Cartesiano e também com os dois tipos de junção oferecidos pela Álgebra Relacional básica: Junção Natural e Junção Theta. Entretanto, a Linguagem SQL oferece outros tipos de junção ainda mais flexíveis e poderosas e, por isso, mas adotadas na prática. São elas:

- Junção Interna: INNER JOIN
- Junções Externas: LEFT JOIN, RIGHT JOIN e FULL JOIN
- NON-EQUIJOIN
- SELF-JOIN

Esta aula apresenta e compara as técnicas de junção relacionadas acima.

II. Junção Interna x Junção Externa

Iniciaremos nosso estudo abordando os conceitos de junção interna (*inner join*) e junção externa (*outer join*) e demonstrando o uso prático destas junções na SQL. Para tal, faremos uso de exemplos baseados em um BD composto por duas tabelas:

Ocupacao

<u>id</u>	nome
1	Engenheiro
2	Desenvolvedor
3	Analista de Dados
4	Minerador de Dados
5	Matemático

Pessoa

<u>cod_pessoa</u>	nome	idade	genero	id_ocup
P01	George	58	M	5
P02	Jane	32	F	3
P03	Aldous	40	M	3
P04	Thomas	28	M	1
P05	Mary	43	F	NULL



Escola Nacional de Ciências Estatísticas

Crie um novo banco de dados SQLite (sugestão de nome: “aula_joins.db”) e execute o script abaixo para criar e popular as duas tabelas:

```
CREATE TABLE Ocupacao(id INTEGER NOT NULL, nome VARCHAR(20) NOT NULL, PRIMARY KEY(id));
```

```
CREATE TABLE Pessoa(cod_pessoa CHAR(3) NOT NULL, nome VARCHAR(60) NOT NULL, idade INTEGER, genero CHAR(1), id_ocup INTEGER, PRIMARY KEY(cod_pessoa), FOREIGN KEY(id_ocup) REFERENCES Ocupacao(id));
```

```
INSERT INTO Ocupacao VALUES (1,'Engenheiro'), (2, 'Desenvolvedor'), (3, 'Analista de Dados'), (4, 'Minerador de Dados'), (5, 'Matemático');
```

```
INSERT INTO Pessoa VALUES ('P01', 'George', 58, 'M', 5), ('P02', 'Jane', 32, 'F', 3), ('P03', 'Aldous', 40, 'M', 3), ('P04', 'Thomas', 28, 'M', 1), ('P05', 'Mary', 43, 'F', NULL);
```

Neste BD, a coluna “id_ocup” da tabela *Pessoa* possui uma referência para “id” da tabela *Ocupacao*. Ou seja, o “id da ocupação” faz a ligação entre as duas tabelas, atuando como chave primária em *Ocupacao* e chave estrangeira em *Pessoa*. Na instância apresentada, observe que existe uma funcionária sem ocupação cadastrada (“P05-Mary”) e também duas profissões que não estão associadas a nenhum funcionário (“2-Desenvolvedor” e “4-Minerador de Dados”).

II.1 Junção Interna (INNER JOIN)

A junção interna (INNER JOIN) combina uma linha de uma tabela *A* com uma linha de uma tabela *B* **apenas quando há coincidência nos valores das colunas** especificadas na **condição de junção**. No exemplo abaixo, o INNER JOIN é utilizado para obter a lista de todos os funcionários e os nomes de suas profissões:

```
SELECT P.*, O.*  
FROM Pessoa P INNER JOIN Ocupacao O ON (P.id_ocup = O.id)  
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro

Para realizar a junção interna entre duas tabelas em SQL, basta seguir sempre a mesma receita, que consiste em:



Escola Nacional de Ciências Estatísticas

1. Especificar os nomes das tabelas envolvidas na junção na cláusula FROM. Cada tabela poderá receber um “apelido”, utilizado para simplificar a referência às mesmas. Em nosso exemplo, as *Pessoa* recebeu o apelido “P” e *Ocupacao* o apelido “O”.
2. Especificar o tipo de junção INNER JOIN entre as duas tabelas.
3. Escrever a palavra-chave ON e depois especificar entre parênteses a condição de junção entre as duas tabelas. Isso significa simplesmente, indicar explicitamente qual é ou quais são o(s) atributo(s) que liga(m) as duas tabelas. No exemplo apresentado, o atributo em comum é o “id da profissão”, que se chama “id” em *Ocupacao* e possui o nome “id_ocup” em *Pessoa*.

A junção final ficou então definida como:

```
FROM Pessoa P INNER JOIN Ocupacao O ON (P.id_ocup = O.id);
```

Algumas informações adicionais:

- A pessoa “P05-Mary” não aparece no resultado final, pois ela não possui um código de profissão associado (está com o valor NULL armazenado na coluna “id_ocup”).
- Na primeira linha do comando SELECT, os campos selecionados foram precedidos pelo apelido de cada tabela, para indicar a que tabela pertence cada coluna (tabela “P” ou tabela “O”). **Os prefixos são obrigatórios apenas para remover ambiguidade** que ocorre quando se deseja projetar alguma coluna de uma tabela cujo nome é igual ao de uma coluna da outra tabela (no exemplo apresentado a coluna “nome”, que existe nas duas tabelas). No entanto, o uso de prefixos em todos os campos (sejam ambíguos ou não) é considerado uma boa prática de programação.
- Tanto faz especificar `ON (P.id_ocup = O.id)` ou `ON (O.id = P.id_ocup)`: de ambas as formas a junção interna será corretamente implementada.
- O `ORDER BY` do exemplo **não** é requerido para viabilizar a junção (ORDER BY não tem nada a ver com JOIN!!!!). Ele foi utilizado apenas para ordenar os resultados pelo código da pessoa, algo que será mantido nos demais exemplos desta seção.

A partir do exemplo apresentado, é possível concluir que o INNER JOIN possui o mesmo propósito do NATURAL JOIN, oferecendo porém a vantagem de **não** exigir que a coluna de ligação possua o mesmo nome nas duas tabelas. Por essa razão, o **INNER JOIN** é que é **realmente utilizado** na prática. Em nosso exemplo, a operação de NATURAL JOIN entre *Pessoa* e *Ocupacao* inclusive falharia, produzindo um conjunto vazio como resultado, uma vez que a coluna que possui nome coincidente em ambas as tabelas é “nome”, que na verdade não representa um atributo de ligação entre as tabelas (em *Pessoa* “nome” é o nome da pessoa e em *Ocupacao* é o nome da profissão).



INNER JOIN: Sintaxe Alternativa (não recomendada)

O exemplo a seguir apresenta uma sintaxe alternativa para o INNER JOIN:

```
SELECT P.*, O.*  
FROM Pessoa P, Ocupacao O  
WHERE P.id_ocup = O.id  
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro

Nesta sintaxe, o **INNER JOIN** é especificado como uma **Junção Theta**. A indicação das tabelas *Pessoa* e *Ocupacao* separadas por vírgula força o produto cartesiano entre ambas as tabelas. Depois, a condição `P.id_ocup = O.id` na cláusula `WHERE` faz com que apenas as linhas onde estas colunas possuam valores coincidentes sejam levadas para o resultado final. Esta sintaxe na verdade faz a junção interna através de uma “junção theta” e não é muito utilizada nos dias atuais, por isso o ideal é evitá-la.

Resumo:

- O INNER JOIN possui o mesmo princípio de funcionamento do NATURAL JOIN, porém é mais flexível, pois não exige que o atributo de ligação possua o mesmo nome nas duas tabelas.
- O INNER JOIN combina duas linhas apenas se elas “casarem”. Ou seja, qualquer linha de *A* que não case com nenhuma linha de *B* e qualquer linha de *B* que não case com uma linha de *A* ficará de fora do resultado final.

II.2 Junção Externa (OUTER JOIN)

As junções externas conectam linhas de duas tabelas de uma forma mais inclusiva: elas produzem os mesmos resultados do INNER JOIN **acrescidos** de linhas “não-casadas” de uma ou ambas as tabelas. As subseções a seguir descrevem os três tipos de junção externa: LEFT JOIN (a mais usada), RIGHT JOIN e FULL JOIN.

II.2.1 LEFT JOIN

A operação de LEFT JOIN (ou LEFT OUTER JOIN) retorna todas as linhas da tabela especificada à esquerda da junção, mesmo que não exista casamento (valor equivalente) na tabela à direita. Para que o conceito fique claro, vamos retornar para o nosso exemplo envolvendo as tabelas *Pessoa* e *Ocupacao*. Neste exemplo, a pessoa ‘P05-Mary’ não possui uma ocupação cadastrada (o campo “id_ocup” possui valor *NULL* para esse registro em *Pessoa*). Se desejarmos produzir um relatório contendo todos os funcionários com seus



Escola Nacional de Ciências Estatísticas

respectivos nomes de profissão, incluindo também os funcionários sem profissão cadastrada, é preciso fazer uso do LEFT JOIN como mostrado abaixo:

```
SELECT P.*, O.*  
FROM Pessoa P LEFT JOIN Ocupacao O ON (P.id_ocup = O.id)  
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro
P05	Mary	43	F	NULL	NULL	NULL

No resultado, observe que as quatro primeiras linhas são as mesmas geradas pelo INNER JOIN. A única linha extra é a quinta, onde os dados da funcionária 'P05' (dados vindo de *Pessoa*) foram combinados com *NULL* (dados que viriam de *Ocupacao* caso ela tivesse alguma cadastrada).

Para realizar a LEFT JOIN entre duas tabelas em SQL, a receita é quase idêntica à utilizada para implementar um INNER JOIN. O único detalhe a ser observado é que, na cláusula FROM, **a tabela a qual desejamos todas as linhas no resultado final deve ser especificada à esquerda na definição da junção**. Como em nosso exemplo, desejávamos todos os funcionários no resultado final, independentemente de terem profissão ou não, por isso a tabela *Pessoa* foi especificada à esquerda da cláusula LEFT JOIN, enquanto *Ocupacao* ficou à direita:

```
FROM Pessoa P LEFT JOIN Ocupacao O ON (P.id_ocup = O.id).
```

Com relação a condição de junção (ON) a ordem não faz diferença, ou seja, tanto faz especificar **ON (P.id_ocup = O.id)** ou **ON (O.id = P.id_ocup)**.

Para listar todos os cargos e funcionários, mesmo os cargos que não tenham um funcionário associado, é preciso especificar *Ocupacao* à esquerda.

```
SELECT P.*, O.*  
FROM Ocupacao O LEFT JOIN Pessoa P ON (P.id_ocup = O.id)  
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
NULL	NULL	NULL	NULL	NULL	2	Desenvolvedor
NULL	NULL	NULL	NULL	NULL	4	Minerador de Dados
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro



Resumo:

- O LEFT JOIN produz os mesmos dados do INNER JOIN, acrescidos das linhas da tabela *A* (tabela à esquerda) que não casam com nenhuma linha de *B* (tabela à direita).
- O LEFT JOIN é o **tipo de junção externa mais utilizado**.

II.2.2 RIGHT JOIN

A junção do tipo RIGHT JOIN (ou RIGHT OUTER JOIN) retorna todas as linhas da tabela à direita, mesmo que não exista casamento (valor equivalente) na tabela à esquerda. Desta forma, para produzir um relatório com todos os funcionários com seus respectivos nomes de profissão, incluindo os funcionários sem profissão cadastrada, seria necessário montar o comando SELECT da forma indicada abaixo:

```
SELECT P.*, O.*  
FROM Ocupacao O RIGHT JOIN Pessoa P ON (P.id_ocup = O.id)  
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro
P05	Mary	43	F	NULL	NULL	NULL

Como o LEFT JOIN e o RIGHT JOIN funcionam da mesma maneira (a única diferença é que no primeiro caso iremos especificar a tabela a qual desejamos todas as linhas no resultado à esquerda e no segundo caso ela ficará à direita), na prática **o LEFT JOIN acabou se tornando mais adotado pelos programadores**. Versões antigas do SQLite sequer aceitavam a utilização do RIGHT JOIN (mas as recentes aceitam)!

Resumo:

- O RIGHT JOIN produz os mesmos dados do INNER JOIN, acrescidos das linhas da tabela *B* (tabela à direita) que não casam com nenhuma linha de *A* (tabela à esquerda).
- O RIGHT JOIN é **bem menos utilizado na prática**. Os desenvolvedores de sistemas preferem utilizar o LEFT JOIN.

II.2.3 FULL JOIN

A junção do tipo FULL JOIN (ou FULL OUTER JOIN) junta em uma única operação os resultados do LEFT JOIN e do RIGHT JOIN. Ela retorna todas as linhas da tabela à esquerda e todas as linhas da tabela à direita devidamente combinadas, de acordo com a especificação da condição de junção. Caso existam linhas na tabela esquerda que não casem



Escola Nacional de Ciências Estatísticas

com qualquer linha da tabela direita, elas serão levadas para o resultado final. E caso existam linhas na tabela direita que não casem com qualquer linha da tabela esquerda, elas também serão levadas para o resultado final.

```
SELECT P.*, O.*
FROM Pessoa P FULL JOIN Ocupacao O ON (P.id_ocup = O.id)
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
NULL	NULL	NULL	NULL	NULL	2	Desenvolvedor
NULL	NULL	NULL	NULL	NULL	4	Minerador de Dados
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro
P05	Mary	43	F	NULL	NULL	NULL

No exemplo anterior, veja que tanto a funcionária sem profissão (linha 5 do resultado), como as profissões que não estão associadas a nenhum funcionário (linhas 1 e 2 do resultado) foram listadas pelo SELECT. É possível reproduzir a funcionalidade do FULL JOIN utilizando LEFT JOIN + UNION, conforme mostrado a seguir:

```
SELECT P.*, O.*
FROM Pessoa P LEFT JOIN Ocupacao O ON (P.id_ocup = O.id)
UNION
SELECT P.*, O.*
FROM Ocupacao O LEFT JOIN Pessoa P ON (P.id_ocup = O.id)
ORDER BY P.cod_pessoa;
```

cod_pessoa	p.nome	idade	genero	id_ocup	id	o.nome
NULL	NULL	NULL	NULL	NULL	2	Desenvolvedor
NULL	NULL	NULL	NULL	NULL	4	Minerador de Dados
P01	George	58	M	5	5	Matemático
P02	Jane	32	F	3	3	Analista de Dados
P03	Aldous	40	M	3	3	Analista de Dados
P04	Thomas	28	M	1	1	Engenheiro
P05	Mary	43	F	NULL	NULL	NULL

Resumo: O FULL JOIN produz os mesmos dados do INNER JOIN, acrescidos das linhas da tabela *A* (tabela à esquerda) que não casam com nenhuma linha de *B* (tabela à direita) e também das linhas de *B* que não casam com nenhuma linha de *A*.

III. Non-Equijoins

Todas as junções apresentadas na seção anterior são do tipo *equijoin*. Isto quer dizer que as junções foram implementadas utilizando **atributos comuns** entre as duas tabelas e as linhas foram combinadas sempre que as colunas possuíam **valores iguais** para tais atributos. Tipicamente, tais atributos estão vinculados através de integridade referencial (PK-FK).



Escola Nacional de Ciências Estatísticas

Entretanto, também é possível combinar linhas de duas tabelas com base em algum outro tipo de **critério diferente da igualdade** (ex: maior, menor ou igual, etc.). Este tipo de junção é chamado de *non-equijoin*. Esta seção introduz o conceito de *non-equijoin* a partir de um exemplo que utiliza as tabelas *T_Aluno*, que armazena os nomes e notas de 3 alunos em uma determinada avaliação; e *T_Conceito*, que contém informação sobre o critério adotado para atribuir conceitos para os alunos (quem tira nota entre 90 e 100 obtém conceito 'A', etc.).

T_Aluno

nome	nota
Rakesh	95
Dayse	50
Keith	72

T_Conceito

conceito	nota_min	nota_max
A	90	100
B	80	89
C	70	79
D	50	69
E	0	49

As tabelas podem ser criadas e populadas com o uso do script abaixo:

```
CREATE TABLE T_Aluno(nome VARCHAR(60) PRIMARY KEY, nota INTEGER);
CREATE TABLE T_Conceito(conceito CHAR(1) PRIMARY KEY, nota_min INTEGER,
nota_max INTEGER);
```

```
INSERT INTO T_Aluno VALUES ('Rakesh', 95), ('Dayse', 50), ('Keith', 72);
```

```
INSERT INTO T_Conceito VALUES ('A', 90, 100), ('B', 80, 89), ('C', 70, 79),
('D', 50, 69), ('E', 0, 49);
```

A seguir, apresenta-se um SQL que realiza um non-equijoin entre estas tabelas para listar o nome, a nota e o conceito de cada aluno:

```
SELECT a.nome, a.nota, c.conceito
FROM T_Aluno a JOIN T_Conceito c
ON (a.nota BETWEEN c.nota_min AND c.nota_max);
```

nome	nota	conceito
Rakesh	95	A
Dayse	50	D
Keith	72	C



Escola Nacional de Ciências Estatísticas

Observe a sintaxe deste tipo de junção. Na segunda linha, a palavra-chave JOIN foi utilizada para indicar que desejávamos realizar a junção entre *T_Aluno* (apelidada de *a*) e *T_Conceito* (apelidada de *c*). Na linha 3, em vez de implementar um *equijoin*, indicamos que uma linha de *a* deveria ser combinada com uma linha de *c* apenas quando o valor da “nota” (coluna de *a*) estivesse compreendido entre os valores de “nota_min” e “nota_max” (colunas de *c*).

Em essência, *non-equijoin* é apenas um nome mais “comercial” para a nossa conhecida **Junção Theta** e que, por essa razão, acabou por ser adotado pelos SGBDs. Trata-se da operação combina linhas de duas tabelas com base em uma condição arbitrária, diferente da igualdade. É até mesmo possível implementar um *non-equijoin* utilizando-se a sintaxe da Junção Theta, conforme mostrado abaixo:

```
SELECT a.nome, a.nota, c.conceito
FROM T_Aluno a, T_Conceito c
WHERE a.nota BETWEEN c.nota_min AND c.nota_max;
```

nome	nota	conceito
Rakesh	95	A
Dayse	50	D
Keith	72	C

Resumo:

- A junção interna (INNER JOIN) e as junções externas (LEFT JOIN, RIGHT JOIN e FULL JOIN) são *equijoins*, pois utilizam a igualdade como critério de comparação.
- Um *non-equijoin* é um tipo de junção baseado em qualquer critério de comparação que não seja a igualdade.
- Um *non-equijoin* pode ser implementado com o uso da palavra-chave JOIN.
- Um *non-equijoin* também pode ser implementado como uma Junção Theta. Tratam-se de conceitos quase que equivalentes (só não podemos dizer que são equivalentes porque a Junção Theta também permite a elaboração de *equijoins*, como foi mostrado na página 5!)

IV. Self-Join

Self-join é o nome dado para um tipo de junção onde as linhas de uma dada tabela são combinadas com outras linhas dela própria, tendo por base algum critério que compara duas diferentes colunas. Para que o conceito fique claro, considere a tabela *T_Filme*, cuja instância é apresentada abaixo.



T_Filme

id	nome	id_pred
1	Guerra nas Estrelas	<i>NULL</i>
2	O Império Contra-Ataca	1
3	O Retorno de Jedi	2
4	Avatar	<i>NULL</i>

Esta tabela possui três campos: “id” (id do filme), “nome” (nome do filme) e “id_pred” (id do filme predecessor, caso faça parte de alguma sequência). Veja que os filmes “Guerra nas Estrelas” e “Avatar” possuem o campo “id_pred” com valor *NULL*, pois eles não representam a sequência de nenhum outro filme. Já “O Império Contra-Ataca” possui “id_pred” com o valor 1, para indicar que ele é um sucessor de “Guerra nas Estrelas” (filme com id=1). De maneira análoga, “O Retorno de Jedi” possui “id_pred” com valor 2, para indicar que se trata de uma sequência de “O Império Contra-Ataca” (filme com id=2).

O script para criar e popular a tabela é apresentado abaixo. Veja que *T_Aluno* possui uma chave estrangeira no campo “id_pred”, que referencia o campo “id”, que é a chave primária da própria tabela.

```
CREATE TABLE T_Filme
(id INT, nome VARCHAR(100), id_pred INT,
PRIMARY KEY(id),
FOREIGN KEY (id_pred) REFERENCES T_Filme(id));

INSERT INTO T_Filme VALUES(1, 'Guerra nas Estrelas', NULL);
INSERT INTO T_Filme VALUES(2, 'O Império Contra-Ataca', 1);
INSERT INTO T_Filme VALUES(3, 'O Retorno de Jedi', 2);
INSERT INTO T_Filme VALUES(4, 'Avatar', NULL);
```

A seguir apresenta-se um exemplo de *self-join* capaz de produzir uma listagem contendo nome de cada filme e o nome de seu predecessor, caso o mesmo exista.

```
SELECT a.nome AS nome_filme, b.nome AS nome_predecessor
FROM T_Filme a LEFT JOIN T_Filme b ON (a.id_pred = b.id);
```

nome_filme	nome_predecessor
Guerra nas Estrelas	<i>NULL</i>
O Império Contra-Ataca	Guerra nas Estrelas
O Retorno de Jedi	O Império Contra-Ataca
Avatar	<i>NULL</i>

Do exemplo apresentado, é possível notar que para implementar um *self-join* torna-se preciso:



1. **Especificar a tabela duas vezes** na cláusula FROM, **atribuindo apelidos diferentes** para cada especificação (em nosso exemplo os apelidos foram *a* e *b*).
2. Definir um critério de JOIN. No exemplo apresentada, utilizou-se o LEFT JOIN para que fosse possível listar tantos os filmes que possuem predecessor, como aqueles que não possuem.
3. Especificar cuidadosamente a condição de junção entre *a* e *b*. Em nosso exemplo, utilizou-se a condição **ON** (*a.id_pred = b.id*) para combinar o id do predecessor de *a* com o id do filme *b*.

Resumo:

- Um *self-join* ocorre quando realiza-se a junção de uma tabela com ela própria.
- Na condição de junção, uma coluna da tabela será combinada com outra coluna diferente da mesma tabela.
- É possível implementar *self-joins* através de qualquer tipo de *equijoin* (INNER, LEFT, RIGHT e FULL) ou de *non-equijoins*.

V. Produto Cartesiano (CROSS JOIN)

Na linguagem SQL, CROSS JOIN é a mesma coisa que produto cartesiano. Ou seja, é uma operação que produz um conjunto de resultados onde cada linha de uma tabela é combinada com cada linha da outra tabela, sem que haja qualquer tipo de critério para realizar a combinação. Desta forma, o CROSS JOIN entre *Pessoa* e *Ocupacao* produz um conjunto de resultados contendo $5 \times 5 = 25$ linhas.

```
SELECT P.*, O.*  
FROM Pessoa P CROSS JOIN Ocupacao O;
```

mat	nome	idade	sexo	id_ocup	id	nome
M01	George	58	M	5	1	Engenheiro
M01	George	58	M	5	2	Desenvolvedor
M01	George	58	M	5	3	Analista de Dados
...						
P05	Mary	43	F	NULL	5	Matemático

Observações:

- O SELECT acima equivale a: **SELECT P.*, O.* FROM Pessoa P, Ocupacao O** (sintaxe que havíamos apresentado para o produto cartesiano na aula 10,).
- Dentro dos aplicativos e sistemas mais convencionais, o produto cartesiano é pouco utilizado, sendo seu uso limitado a situações especiais.



Escola Nacional de Ciências Estatísticas

- Um comando SELECT contendo uma condição de JOIN especificada de forma errada normalmente produzirá como resultado um produto cartesiano entre as duas tabelas envolvidas. Por este motivo, muitos desenvolvedores de sistema associam o termo “produto cartesiano” ao significado “junção errada”. Mas isso não é verdade! O produto cartesiano é uma operação básica da Álgebra Relacional que pode ser necessária em algumas situações especiais.

VI. Junção de Múltiplas Tabelas

Todos os exemplos apresentados nesta aula envolveram a junção entre duas tabelas. Porém, é possível montar comandos SELECT contendo diversos INNER JOINs, LEFT JOINs, etc. Neste caso, o importante é saber que a operação de junção será sempre **feita aos pares** (resolvendo duas relações de cada vez), respeitando a ordem em que estes pares são especificados (diversos exemplos serão apresentados nas próximas aulas). Vamos ver como isto funciona na prática com o uso de um exemplo. Considere o quadro seguir, que apresenta o esquema do BD do sistema de RH uma empresa e, em seguida, um script para definir as tabelas e inserir alguns dados de exemplo.

Nome da Relação	Descrição
<i>Departamento</i> (<u>codDepto</u> , nomDepto)	Departamentos da empresa.
<i>Funcionario</i> (<u>matFunc</u> , nome, idade, salario, codDepto)	Matrícula (“matFunc”), nome, idade, salário e código do departamento (“codDepto”) dos funcionários da empresa.
<i>Projeto</i> (<u>codProj</u> , nomProj, uf)	Projetos da empresa. Cada projeto tem um código identificador único (“codProj”), um nome (“nomProj”) e é conduzido em alguma UF do Brasil (ex: 'RJ', 'DF', 'MG', ...)
<i>EquipeProjeto</i> (<u>matFunc</u> , <u>codProj</u>)	Armazena os funcionários alocados a cada projeto.

```
CREATE TABLE Departamento(codDepto INT NOT NULL PRIMARY KEY, nomDepto
VARCHAR(15) NOT NULL);
```

```
INSERT INTO Departamento VALUES(1, 'Engenharia'), (2, 'Arquitetura'), (3,
'RH');
```

```
CREATE TABLE Funcionario(matFunc CHAR(2) NOT NULL PRIMARY KEY, nome
VARCHAR(60) NOT NULL, idade INT NOT NULL, salario FLOAT NOT NULL, codDepto
INT NOT NULL, FOREIGN KEY (codDepto) REFERENCES Departamento(codDepto));
```

```
INSERT INTO Funcionario VALUES('M1', 'Jennifer', 30, 5000, 1);
INSERT INTO Funcionario VALUES('M2', 'Hector', 22, 3000, 3);
INSERT INTO Funcionario VALUES('M3', 'Molina', 49, 4200, 1);
INSERT INTO Funcionario VALUES('M4', 'Larry', 25, 1500, 2);
INSERT INTO Funcionario VALUES('M5', 'Ellison', 37, 4200, 1);
INSERT INTO Funcionario VALUES('M6', 'Yuko', 31, 3000, 2);
```



Escola Nacional de Ciências Estatísticas

```
CREATE TABLE Projeto(codProj INT NOT NULL PRIMARY KEY, nomProj VARCHAR(25) NOT NULL, uf CHAR(2) NOT NULL);
```

```
INSERT INTO Projeto VALUES(50, 'Projeto Alpha', 'RJ'), (99, 'Projeto Beta', 'DF');
```

```
CREATE TABLE EquipeProjeto(matFunc CHAR(2) NOT NULL, codProj INT NOT NULL, PRIMARY KEY (matFunc, codProj), FOREIGN KEY (matFunc) REFERENCES Funcionario(matFunc), FOREIGN KEY (codProj) REFERENCES Projeto(codProj));
```

```
INSERT INTO EquipeProjeto VALUES('M1', 50), ('M3', 50), ('M4', 50), ('M1', 99), ('M5', 99);
```

Departamento

codDepto	nomDepto
1	Engenharia
2	Arquitetura
3	RH

Funcionario

matFunc	nome	idade	salario	codDepto
M1	Jennifer	30	5000	1
M2	Hector	22	3000	3
M3	Molina	49	4200	1
M4	Larry	25	1500	2
M5	Ellison	37	4200	1
M6	Yuko	31	3000	2

Projeto

codProj	nomProj	uf
50	PROJETO ALPHA	RJ
99	PROJETO BETA	DF

EquipeProjeto

matFunc	codProj
M1	50
M3	50
M4	50
M1	99
M5	99



Escola Nacional de Ciências Estatísticas

Exemplo 1: Recuperar a matrícula e nome dos funcionários e o nome do departamento onde os mesmos trabalham..

```
SELECT f.matFunc, f.nome, d.nomDepto
FROM Funcionario f INNER JOIN Departamento d ON (f.codDepto = d.codDepto)
ORDER BY f.matFunc;
```

Resultado Final

matFunc	nome	nomDepto
M1	Jennifer	Engenharia
M2	Hector	RH
M3	Molina	Engenharia
M4	Larry	Arquitetura
M5	Ellison	Engenharia
M6	Yuko	Arquitetura

Neste caso, foi preciso realizar a junção entre as relações *Funcionario* (que contém o nome e matrícula do funcionário) e *Departamento* (que contém o nome do departamento). A junção entre as tabelas é realizada pelo atributo em comum, o código do departamento (“codDepto”).

Exemplo 2: Recuperar a matrícula e nome dos funcionários, o nome do departamento onde os mesmos trabalham e também o(s) código(s) dos projetos em que cada funcionário está alocado.

```
SELECT f.matFunc, f.nome, d.nomDepto, e.codProj
FROM Funcionario f
INNER JOIN EquipeProjeto e ON (f.matFunc = e.matFunc)
INNER JOIN Departamento d ON (f.codDepto = d.codDepto)
ORDER BY f.matFunc;
```

Resultado Final

matFunc	nome	nomDepto	codProj
M1	Jennifer	Engenharia	50
M1	Jennifer	Engenharia	99
M3	Molina	Engenharia	50
M4	Larry	Arquitetura	50
M5	Ellison	Engenharia	99



Escola Nacional de Ciências Estatísticas

A seguir a explicação sobre como essa consulta é **resolvida internamente** pelo SGBD:

PASSO 1: Conforme mencionado anteriormente, as junções são **sempre feitas aos pares e seguindo a ordem especificada**. Sendo assim, primeiro realiza-se a junção interna entre *Funcionario* e *EquipeProjeto* gerando um **resultado intermediário** (uma tabela temporária na memória de trabalho do SGBD) que é apresentado a seguir. Neste resultado intermediário, teremos um conjunto contendo todas as tuplas de *Funcionario* corretamente “casadas” com as tuplas de *EquipeProjeto*.

Resultado intermediário 1 – produzido pela junção de *Funcionario* com *EquipeProjeto*

matFunc	nome	idade	salario	codDepto	matFunc:	codProj
M1	Jennifer	30	5000	1	M1	50
M3	Molina	49	4200	1	M3	50
M4	Larry	25	1500	2	M4	50
M1	Jennifer	30	5000	1	M1	99
M5	Ellison	37	4200	1	M5	99

Algumas observações **importantes**:

- Conceitualmente, a operação de junção **não descarta** nenhum atributo do par de tabelas envolvidas. Por isso, o resultado intermediário contém todos os atributos de *Funcionario* e todos os de *EquipeProjeto*.
- Observe que: (i) os funcionários de matrícula ‘M2’ e ‘M6’ não aparecem no resultado, pois não estão alocados a nenhum projeto; (ii) a funcionária de matrícula ‘M1’ aparece duas vezes, pois ela está alocada a dois projetos.

PASSO 2: no passo seguinte, o conjunto de resultados intermediário será então utilizado na junção natural com a relação *Departamento*. Como resultado (ainda um resultado intermediário), teremos um conjunto contendo todas as tuplas de *Resultado intermediário 1* corretamente casadas com as tuplas de *Departamento*. O atributo em comum entre ambas as relações desta vez é “codDepto” e o atributo que é acrescentado é “NomDepto”.

Resultado intermediário 2 – produzido pela junção do *Resultado intermediário 1* com *Departamento*

matFunc	nome	idade	salario	codDepto	matFunc:	codProj	codDepto:	nomDepto
M1	Jennifer	30	5000	1	M1	50	1	Engenharia
M3	Molina	49	4200	1	M3	50	1	Engenharia
M4	Larry	25	1500	2	M4	50	2	Arquitetura
M1	Jennifer	30	5000	1	M1	99	1	Engenharia
M5	Ellison	37	4200	1	M5	99	1	Engenharia



Escola Nacional de Ciências Estatísticas

PASSO 3: por fim, estando todas as junções resolvidas, o SGBD gera o resultado final projetando os atributos que o usuário especificou no operador : matFunc, nome, nomDepto, codProj.

Resultado Final

matFunc	nome	nomDepto	codProj
M1	Jennifer	Engenharia	50
M1	Jennifer	Engenharia	99
M3	Molina	Engenharia	50
M4	Larry	Arquitetura	50
M5	Ellison	Engenharia	99

Exemplo 3: Modificar a consulta do exemplo anterior para exibir o nome do projeto ao invés do código.

```
SELECT f.matFunc, f.nome, d.nomDepto, p.nomProj
FROM Funcionario f
INNER JOIN EquipeProjeto e ON (f.matFunc = e.matFunc)
INNER JOIN Projeto p ON (e.codProj = p.codProj)
INNER JOIN Departamento d ON (f.codDepto = d.codDepto)
ORDER BY f.matFunc;
```

Resultado Final

matFunc	nome	nomDepto	nomProj
M1	Jennifer	Engenharia	Projeto Alpha
M1	Jennifer	Engenharia	Projeto Beta
M3	Molina	Engenharia	Projeto Alpha
M4	Larry	Arquitetura	Projeto Alpha
M5	Ellison	Engenharia	Projeto Beta

Neste exemplo, a tabela *Projeto* precisou ser acrescentada à consulta, pois o nome do projeto (“nomProj”) está armazenado nela. Como não existem atributos em comum entre *Funcionario* e *Projeto*, primeiro é preciso realizar a junção de *Funcionario* com *EquipeProjeto* (possuem “matFunc” como atributo em comum). O resultado produzido poderá então sofrer uma junção com *Projeto* (possuirão “codProj” como variável em comum).

IMPORTANTE: Como observação final é importante comentar que não apenas as junções interna e externa, mas **também a junção theta e o produto cartesiano** são resolvidos aos pares, respeitando a ordem de especificação dos mesmos dentro da consulta.